



Agentic Open-World 3D Scene Generation at Scale

Tencent Hunyuan3D



Abstract

Generating large-scale, freely explorable 3D worlds from open-ended text remains challenging because a system must jointly maintain global spatial coherence, rich local content, and explicit assets suitable for downstream editing and reuse. We present WorldClaw, a fully agentic, coarse-to-fine framework for open-world 3D scene generation. Planning agents translate a text prompt into a structured specification of regions, terrain, assets, materials, and spatial relations. WorldClaw then builds a globally coherent terrain foundation from semantic layouts, reusable assets, generative or procedural materials, and a region-aware height field. For detail-demanding regions, it generates terrain-conditioned compositions, reconstructs editable textured meshes, and recovers their placement on the terrain; render-based agents further refine terrain, objects, appearance, and contacts. Across diverse open-world prompts, WorldClaw produces large-scale scenes with coherent spatial organization, visually compelling local content, and editable instance-level assets while preserving a consistent global terrain structure.

Date: July 21, 2026

Project Page: <https://github.com/Tencent/Hunyuan3D-WorldClaw>

1 Introduction

Generating freely explorable 3D worlds from a single sentence remains one of the most compelling yet difficult goals in generative content creation. Video games, film production, virtual reality, embodied intelligence, and robot simulation all rely on explicit 3D scenes that preserve coherent geometry, appearance, and semantics throughout a continuous space. Unlike an image or a video locked to a predefined camera path, such a world must not only look plausible, but also be walkable, structurally consistent, and editable. Together, these properties turn generated content from a fixed visual artifact into a persistent, reusable environment that supports exploration, interaction, simulation, and iterative creation.

Open-world 3D scene generation has been studied extensively for decades and has recently gained renewed attention with the emergence of *3D world models*, which aim to create explorable, immersive visual experiences. Nevertheless, generating large-scale 3D scenes with rich, diverse, and high-quality content remains a challenge. We categorize the existing methods into four categories: Procedural Content Generation (PCG), Image/Video-Lifting, Native 3D Diffusion, and Multi-Modal LLM Agents (MLLM-Agents) based methods. **PCG** builds terrain, vegetation, and buildings with rules and executable programs, offering strong controllability and scalability as seen in InfiniteGen [?]. The PCG methods are often constrained by limited representation capacity, as they rely on hard-coded rules and programs to generate content, which is limited by the expressiveness of the rules and programs. **Image/Video-Lifting** methods leverage image or video generators to synthesize scene observations and lift them into 3D representations such as meshes or 3D Gaussians via depth estimation or multi-view reconstruction. Notable products include Marble [?]. Video-generation models offer rich content but often lack global consistency and geometric fidelity. They are also computationally inefficient: for example, generating a complete 3D house scene requires a long video clip that circles the house to capture multiple views, resulting in the generation of many frames. Otherwise, novel viewpoints not covered by the generated video tend to exhibit blurring and incoherent geometry. **Native 3D Diffusion** methods learn scene distributions directly over 3D voxels, distance fields, or Gaussians, improving geometric fidelity and cross-view consistency [? ? ?]. They can efficiently generate high-quality geometry and textures in a single diffusion rollout, but their content diversity is limited by the scarcity of large-scale 3D scene datasets. **MLLM Agents** leverage the world knowledge of multimodal large language models for intent understanding, spatial planning, tool orchestration, and render-based refinement, enabling natural-language instructions to drive executable scene-construction pipelines [? ? ? ?]. However, despite excelling at global planning and semantic-level task execution, they lack precise 3D spatial control. For example, when asked to adjust a single object’s position, they may overcorrect because of inadequate spatial scale awareness, limiting their ability to generate scenes with complex object relationships.

To address these challenges, we present **WorldClaw**, a coarse-to-fine agentic framework for generating explorable 3D worlds from open-ended text. Its design is guided by a central insight: a globally coherent world need not be generated everywhere at once. Shared constraints, including scene semantics, spatial organization, and the terrain foundation, can first be established globally, while the objects and local relationships that distinguish individual regions are progressively realized where needed. Following this global-to-regional principle, WorldClaw proceeds through three stages: *intent analysis and planning* (Section 2.1), *global terrain generation* (Section 2.2), and *regional object generation and placement* (Section 2.3). It first transforms the user prompt into a structured scene specification describing regions, terrain, objects, appearance, and spatial relationships; it then constructs an irregular terrain with explicit regional semantics to anchor scene scale and spatial layout; and finally, it generates, places, and refines instance-level content in regions that require further development. Throughout this process, specialized agents translate high-level intent into executable plans, orchestrate the required procedural and generative tools, and inspect and refine the resulting terrain and objects.

At the global terrain generation stage, WorldClaw treats terrain as the organizing geometry of the scene. Guided by a structured terrain plan and semantic layout map, it combines region-aware procedural terrain construction with generated materials and reusable assets to create irregular, regionally organized landforms. Render-based refinement further improves geometry and appearance, yielding a controllable global foundation for subsequent regional content generation.

After establishing the global terrain, WorldClaw selectively populates only regions that require instance-level

content. For each region, it generates a terrain-conditioned composition to organize object appearance and spatial relationships, then decomposes and reconstructs the composition into independently editable 3D instances with terrain-aligned placements. Regional object generation and placement thereby introduce rich, spatially related content while preserving the global structure, and subsequent agentic refinement further improves object quality and object-terrain contact.

The main contributions of this work are summarized as follows:

- We propose a semantic-layout-guided procedural terrain generator that creates controllable, region-aware landforms across spatial scales.
- We translate global scene and local terrain constraints into executable regional plans that yield diverse, spatially coherent object layouts with fine-grained regional control.
- We introduce an agentic refinement loop that iteratively improves terrain and object quality, corrects object scale and pose, and resolves object-terrain contact issues.
- WorldClaw represents scenes as independently editable textured meshes with explicit terrain placements, supporting free-viewpoint rendering, asset reuse, and conventional game-engine workflows.

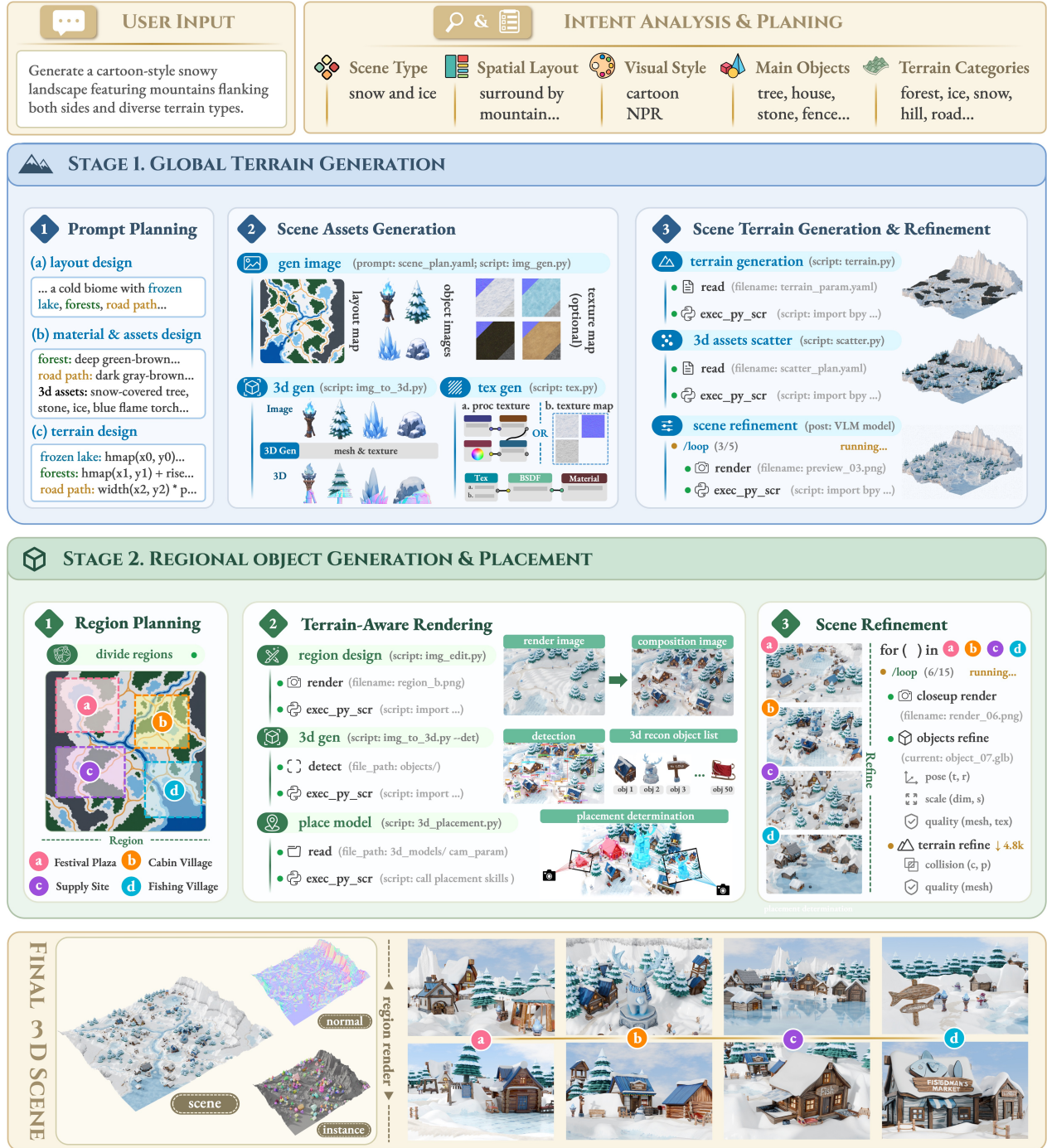


Figure 1 Overview of WorldClaw. Given an open-ended text prompt, WorldClaw constructs an explicit, explorable, and editable 3D world through a three-stage global-to-regional pipeline. (1) *Intent Analysis and Planning* translates the prompt into a structured scene specification. (2) *Global Terrain Generation* establishes a region-aware terrain foundation with coherent geometry, appearance, and spatial semantics. (3) *Regional Object Generation and Placement* selectively populates planned regions with editable 3D assets and refines their arrangements and interactions with the terrain.

2 Method

Given an open-ended user prompt q , WorldClaw aims to construct an explicit 3D world $\mathcal{S}$ whose global organization, terrain geometry, surface appearance, and object composition are consistent with the user intent. The resulting world should support free-viewpoint exploration and preserve terrain and object instances as editable 3D content. Rather than synthesizing the entire world in a single pass, WorldClaw adopts a coarse-to-fine, global-to-regional construction strategy. It first establishes the scene-wide semantics, regional organization, and terrain foundation, and then selectively realizes detailed instance-level content in regions that require further development. As illustrated in Fig. 1, the construction process consists of three sequential stages:

$$\mathcal{P} = F_{\text{plan}}(q), \quad \mathcal{T} = F_{\text{terrain}}(\mathcal{P}), \quad \mathcal{O} = F_{\text{region}}(\mathcal{P}, \mathcal{T}), \quad (1)$$

where $\mathcal{P}$ denotes the structured scene specification, $\mathcal{T}$ denotes the global terrain representation, and $\mathcal{O}$ denotes the set of generated and placed regional object instances. The final world is composed as

$$\mathcal{S} = \text{Compose}(\mathcal{T}, \mathcal{O}). \quad (2)$$

The *intent analysis and planning* stage (Section 2.1) converts q into $\mathcal{P}$, which specifies the regions, terrain requirements, object categories, appearance attributes, and spatial relationships of the target world. The *global terrain generation* stage (Section 2.2) instantiates the terrain and regional constraints in $\mathcal{P}$ as a coherent terrain foundation $\mathcal{T}$. Conditioned on both $\mathcal{P}$ and $\mathcal{T}$, the *regional object generation and placement* stage (Section 2.3) constructs the object set $\mathcal{O}$, determines instance-level placements, and refines local object-terrain relationships. Specialized agents execute the three stages and communicate through shared structured intermediate representations, allowing local generation and refinement to preserve the globally established scene organization.

2.1 Intent Analysis and Planning

Users typically describe a target scene using a short, open-ended text prompt, whereas constructing a complete 3D outdoor scene requires substantially more detailed specifications, including terrain structure, regional organization, object composition, material style, environmental atmosphere, and spatial relationships. Directly passing the raw prompt to downstream generation modules introduces two major challenges. First, users often specify only a small number of salient concepts while omitting the spatial, geometric, and appearance attributes required for scene construction. Second, category references, qualitative spatial relationships, and high-level style descriptions may be ambiguous and interpreted inconsistently across different generation stages. Inspired by prior work [? ? ? ?], WorldClaw introduces an intent analysis and planning module composed of an intent analysis agent and a scene planning agent. Together, they convert the open-ended user prompt into a structured scene specification $\mathcal{P}$, which serves as the shared semantic interface for subsequent terrain construction and regional object generation.

Intent Analysis. Given the user prompt q , the intent analysis agent extracts and normalizes the constraints explicitly expressed by the user, including the scene type, theme and visual style, key regions and objects, spatial relationships, and user preferences. This stage only summarizes information already contained in the prompt and neither introduces new scene content nor completes unspecified attributes. By separating explicit constraint extraction from missing-information completion, it preserves the original user intent and provides reliable constraints for subsequent scene planning.

Scene-Level Planning. Conditioned on the original prompt and its explicit constraints, the scene planning agent resolves ambiguous descriptions while preserving the user requirements and completes unspecified information required by downstream generation modules according to a predefined scene-specification schema. The resulting scene plan is organized as

$$\mathcal{P} = (\mathcal{R}, \mathcal{C}_{\text{terrain}}, \mathcal{C}_{\text{object}}), \quad (3)$$

where $\mathcal{R}$ describes the major scene regions, their attributes, and their spatial relationships. The terrain specification $\mathcal{C}_{\text{terrain}}$ describes the terrain types, landform characteristics, surface appearance, and terrain-associated assets required by each region, while the object specification $\mathcal{C}_{\text{object}}$ specifies the required object categories, appearance attributes, approximate densities, and region-level spatial relationships. Global properties, including the scene theme, overall visual style, material preferences, and environmental atmosphere, are retained as shared attributes of the corresponding fields, ensuring that terrain and object generation follow consistent visual and semantic conditions.

This stage produces the structured scene specification required by subsequent scene construction, rather than directly generating terrain geometry or object instances. The resulting specification $\mathcal{P}$ serves as a shared set of semantic and spatial constraints, ensuring a consistent interpretation of the user intent, regional organization, visual style, and scene content across downstream modules.

2.2 Global Terrain Generation

Terrain is not only the geometric foundation of a 3D world, but also a global structure that organizes regional semantics, spatial hierarchy, surface appearance, and environmental content. Large-scale landforms determine the overall world shape, perceived scale, and explorable extent, while local relief and surface details further affect navigability, visual realism, and stylistic expression. However, many existing scene-generation pipelines simplify the ground as a planar or weakly varying support and organize scene content primarily by placing objects on top of it. Such planar assumptions cannot adequately represent geometrically expressive landforms, including mountains, canyons, dunes, and terraces, and limit the continuous yet semantically distinct organization of different regions.

WorldClaw therefore introduces a global terrain generation module that transforms the structured scene specification $\mathcal{P}$ produced by intent analysis and planning into a global terrain representation $\mathcal{T}$. The module progressively realizes the terrain requirements through explicit, inspectable, and editable intermediate representations. It comprises three stages: terrain planning refines high-level semantic constraints into an executable terrain specification; terrain asset generation constructs a semantic layout map, reusable terrain-asset prototypes, and surface materials; and terrain generation and refinement instantiates these conditions as a renderable 3D terrain with region-aware semantics, composite landforms, surface appearance, and environmental details. The resulting $\mathcal{T}$ provides a shared geometric foundation and spatial semantics for subsequent regional content generation in Section 2.3.

2.2.1 Terrain Planning

The structured scene specification $\mathcal{P}$ describes the regional composition, spatial relationships, and high-level terrain requirements of the target world, but does not yet provide all information required for direct terrain construction. For example, it may not fully specify region boundaries, world scale, landform parameters, asset densities, or material implementations. Directly generating terrain from such high-level constraints would require a single generative process to simultaneously perform semantic interpretation, missing-information completion, and spatial and geometric instantiation. Since the same semantic description may admit multiple plausible terrain designs, the result can appear visually reasonable while deviating from the desired regional relationships, landform hierarchy, or content composition. To bridge the gap between high-level scene semantics and executable terrain construction, we employ a terrain planning agent to produce a structured terrain specification.

Structured Terrain Specification. The terrain planning agent converts the terrain-related constraints in $\mathcal{P}$ into a structured terrain specification

$$\mathcal{P} = (\mathbf{p}_{\text{layout}}, \mathbf{p}_{\text{asset}}, \mathbf{p}_{\text{material}}, \boldsymbol{\theta}_{\text{terrain}}), \quad (4)$$

where $\mathbf{p}_{\text{layout}}$ specifies region categories, relative positions, adjacency relationships, and approximate coverage; $\mathbf{p}_{\text{asset}}$ describes terrain-associated asset categories, regional affinities, and target densities; $\mathbf{p}_{\text{material}}$ defines the surface types, visual styles, and texture requirements of different regions; and $\boldsymbol{\theta}_{\text{terrain}}$ contains numerical parameters such as world scale, region-specific base elevations, noise frequencies and amplitudes, geomorphic

operators and their weights, and boundary-blending widths. We define a standardized schema for these components so that $\mathcal{P}$ serves as an explicit interface among terrain planning, asset generation, and geometric construction.

Tool-Augmented Planning. When the scene specification contains concepts that require external knowledge, the agent invokes a search tool to retrieve relevant references. When textual constraints are insufficient to express complex spatial relationships or visual styles, the agent additionally generates a scene concept image $\mathbf{I}_{\text{concept}}$ as visual conditioning for landform composition, regional layout, and artistic style. The retrieved references and concept image supplement $\mathcal{P}_{\text{terrain}}$ without replacing explicit user constraints; $\mathbf{I}_{\text{concept}}$ is subsequently used as an optional condition for terrain asset generation and regional planning.

2.2.2 Terrain Asset Generation

To convert the textual terrain specification into explicit visual and geometric conditions, the terrain asset generation agent takes $\mathcal{P}_{\text{terrain}}$ and the optional concept image $\mathbf{I}_{\text{concept}}$ as conditions and produces

$$\mathcal{A}_{\text{terrain}} = (\mathbf{I}_{\text{layout}}, \mathcal{I}_{\text{asset}}, \mathcal{O}_{\text{asset}}, \mathcal{M}_{\text{terrain}}), \quad (5)$$

where $\mathbf{I}_{\text{layout}}$ is a scene layout map that constrains regional distributions, $\mathcal{I}_{\text{asset}}$ is a set of representative asset images, $\mathcal{O}_{\text{asset}}$ is a set of 3D assets intended for subsequent global scattering, and $\mathcal{M}_{\text{terrain}}$ is a set of materials defining surface appearance. These intermediate assets are constructed using image, 3D, and material generation capabilities and serve as explicit inputs to subsequent terrain construction. Both $\mathbf{I}_{\text{layout}}$ and $\mathcal{I}_{\text{asset}}$ are generated by GPT-Image-2 [?].

Scene Layout Map. Indoor-scene generation commonly represents spatial layouts using floor plans or structured room representations [?]. These representations are designed primarily for regular walls and approximately planar regions and are therefore not directly suited to natural terrain with curved boundaries and irregular shapes. We instead generate a global semantic layout map $\mathbf{I}_{\text{layout}}$ from $\mathbf{p}_{\text{layout}}$, using distinct colors to encode predefined terrain categories. The layout map converts textual descriptions of region categories, relative positions, adjacency relationships, and coverage into a unified 2D spatial partition, which is subsequently used for region-mask extraction, height-field generation, material assignment, and asset scattering.

Terrain Asset Prototypes. For environmental elements that are repeatedly instantiated across the global terrain, this stage generates reusable 3D asset prototypes without determining instance-specific positions, scales, or orientations. Given the asset requirements $\mathbf{p}_{\text{asset}}$, the agent first generates a representative image set $\mathcal{I}_{\text{asset}}$, including terrain-associated categories such as rocks, vegetation clusters, and landform attachments. The system then uses the image-to-3D capability of Hunyuan3D[?] to convert these references into the reusable 3D asset-prototype set $\mathcal{O}_{\text{asset}}$, which provides geometric candidates for subsequent environmental scattering.

Terrain Materials. To balance expressive local surfaces with scalable large-area coverage, we employ two complementary pathways, namely generative texture synthesis and procedural material generation, to construct $\mathcal{M}_{\text{terrain}}$ according to $\mathbf{p}_{\text{material}}$. The generative pathway produces texture channels such as albedo, normal, and roughness maps for local surfaces with complex appearance or irregular details. The procedural pathway programmatically assembles Blender material nodes to create tileable and parameter-adjustable surface materials for large-scale regions. Materials from both pathways are associated with the semantic regions in $\mathbf{I}_{\text{layout}}$, ensuring consistency between regional organization and surface appearance.

2.2.3 Terrain Generation & Refinement

Given the structured terrain specification $\mathcal{P}_{\text{terrain}}$ and the intermediate asset package $\mathcal{A}_{\text{terrain}}$, the terrain generation agent constructs the global terrain representation $\mathcal{T}$. The representation comprises terrain geometry, regional semantics, surface materials, and scattered terrain-associated assets. The process consists of initial height-field generation, global terrain asset scattering, and terrain refinement.

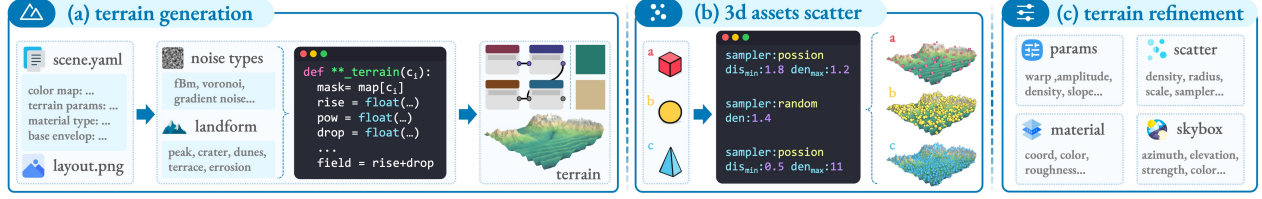


Figure 2 Overview of global terrain generation and refinement. (a) *Initial Height-Field Generation* constructs composite terrain geometry from the semantic layout map and region-specific terrain parameters and assigns materials to the corresponding regions. (b) *Global Terrain Asset Scattering* instantiates terrain-associated assets according to regional semantics and local surface conditions. (c) *Terrain Refinement* renders, inspects, and locally edits the terrain to correct geometric transitions, material scales, asset distributions, and rendering artifacts.

Initial Height-Field Generation. As illustrated in Fig. 2(a), the agent parses the scene layout map I_{layout} , maps its color codes to predefined terrain categories, and extracts a mask for each region r . Boundary smoothing produces normalized soft weights $\tilde{m}_r$, allowing height-field generation, material assignment, and subsequent asset scattering to share the same semantic partition. Given the region-level terrain parameters θ_{terrain} , the global height field is represented as

$$H(\mathbf{x}) = \sum_r \tilde{m}_r(\mathbf{x}) \left[h_r + \sum_k w_{r,k} N_{r,k}(\mathbf{x}) + \sum_j \alpha_{r,j} G_{r,j}(\mathbf{x}) \right], \quad (6)$$

where $\mathbf{x}$ denotes a 2D position in the terrain domain, h_r is the base elevation of region r , $N_{r,k}$ is a noise component at a particular spatial frequency, $G_{r,j}$ is a geomorphic operator such as peak, dune, terrace, or erosion, and $w_{r,k}$ and $\alpha_{r,j}$ control the respective contributions of noise and geomorphic operators [?]. Region-specific composition allows different landforms to form a continuous composite height field with irregular boundaries, while explicit scale parameters allow the same construction logic to accommodate different spatial extents. The same regional weights are subsequently used to assign and blend materials from $\mathcal{M}_{\text{terrain}}$ over the corresponding surfaces.

Global Terrain Asset Scattering. After establishing the base terrain geometry, the system supplements it with mid-scale environmental details using reusable prototypes from $\mathcal{O}_{\text{asset}}$, as illustrated in Fig. 2(b). This stage handles only basic elements closely associated with terrain and ecological distributions, such as rocks, vegetation clusters, and landform attachments. Objects with explicit functions, instance identities, or complex spatial relationships are deferred to the subsequent regional object generation stage. For each terrain-asset category, the agent samples candidate locations within the corresponding layout masks according to the regional affinities and target densities in $\mathbf{p}_{\text{asset}}$. Local elevation, slope, and surface normals are then used to filter or adjust the sampled instances. Their final scales and orientations are adapted to the local surface to reduce floating, penetration, and unnatural distributions.

Terrain Refinement. As illustrated in Fig. 2(c), the initial terrain may still contain abrupt regional transitions, inconsistent landform scales, mismatched texture proportions, unnatural asset distributions, or local rendering artifacts. Powered by BlenderMCP [?], the terrain refinement agent re-renders the scene from predefined viewpoints, inspects its geometry, materials, scattering results, and rendering configuration, and performs localized corrections according to the detected issues. Editable variables include region-specific terrain parameters, boundary blending, material texture scales, asset densities and transforms, and, when necessary, environment lighting and rendering settings. The loop continues until no substantial issue is detected or a predefined iteration budget is reached, improving local geometry and visual quality while preserving the semantic layout and global spatial organization.

2.3 Regional Object Generation and Placement

Section 2.2 constructs a global terrain with explicit region-level semantics, but primarily focuses on large-scale landforms, basic surface appearance, and terrain-associated elements, leaving fine-grained scene objects with

explicit functions and spatial relationships uninstantiated. To populate selected regions while preserving the global terrain structure, we introduce a regional object generation and placement module. The module combines the scene specification $\mathcal{P}_{\text{scene}}$ from Section 2.1 with the generated terrain $\mathbf{T}_{\text{terrain}}$, and plans, generates, and edits only those regions that require further development. This regionalized formulation avoids processing the complete scene in a single pass and provides fine-grained control over regional functions, object compositions, spatial layouts, and visual styles. The overall process comprises regional planning, object generation and placement, and scene refinement.

2.3.1 Regional Planning

The region set $\mathcal{R}$ and object requirements $\mathcal{C}_{\text{object}}$ in the scene specification describe the global regional organization and high-level object requirements, but do not yet determine which regions should be refined or provide executable regional object configurations. The regional planning agent therefore jointly examines $\mathcal{P}_{\text{scene}}$ and $\mathbf{T}_{\text{terrain}}$, prioritizes regions with uninstantiated object requirements whose local terrain can support the requested functions, selects a subset $\mathcal{R}^+ \subseteq \mathcal{R}$ for further development, and refines the scene-level constraints into

$$\mathcal{P}_{\text{regional}} = \left\{ (r, \phi_r, \mathcal{C}_r^{\text{object}}, \mathbf{p}_r^{\text{spatial}}, \mathbf{p}_r^{\text{appearance}}) \mid r \in \mathcal{R}^+ \right\}, \quad (7)$$

where each tuple corresponds to a selected region $r \in \mathcal{R}^+$: ϕ_r denotes its functional role, $\mathcal{C}_r^{\text{object}}$ specifies the categories, counts, and densities of objects to be introduced, $\mathbf{p}_r^{\text{spatial}}$ describes object-object and object-terrain relationships, and $\mathbf{p}_r^{\text{appearance}}$ defines region-level appearance and style requirements. Regional planning completes only the region-level information that remains uninstantiated in the scene specification, while preserving the existing region semantics and relationships in $\mathcal{R}$. When textual constraints are insufficient to specify the desired appearance, the optional concept image $\mathbf{I}_{\text{concept}}$ supplements the regional specification as a visual reference without replacing explicit constraints derived from the user intent. Consequently, $\mathcal{P}_{\text{regional}}$ serves as an explicit interface between global scene planning and subsequent object generation and placement.

2.3.2 Object Generation & Placement

To support controllable object generation and placement over the generated terrain, we decompose this stage into three sequential and reusable skills: *region composition*, *object generation*, and *object placement*. Region composition produces a terrain-conditioned 2D layout prior, object generation separates and reconstructs individual instances from the composition image, and object placement recovers their positions, scales, and orientations in the terrain coordinate system. For each region r , this stage produces an object set

$$\mathcal{O}_r = \left\{ (M_i, \mathcal{U}_i, T_{\text{place}}^i) \right\}_{i=1}^{n_r}, \quad (8)$$

where M_i , $\mathcal{U}_i$, and T_{place}^i denote the geometry, appearance attributes, and placement transformation of object i , respectively. By coordinating the three skills, the regional agent progressively converts $\mathcal{P}_r$ into a 3D object layout consistent with both the synthesized composition and the underlying terrain, while retaining an instance-level representation for subsequent editing and refinement.

Region Composition. For each selected region r , the agent first renders the existing terrain and records the associated camera parameters $\kappa_r = (K_t, E_t)$, producing a terrain image $\mathbf{I}_r^{\text{terrain}}$. In contrast to directly synthesizing an isolated regional image, this rendering preserves local topography, material appearance, viewing direction, and surrounding spatial context. The agent then converts $\mathcal{P}_r$ into a structured regional prompt and generates a region composition image conditioned on the terrain rendering, regional specification, and optional concept image:

$$\mathbf{I}_r^{\text{comp}} = \mathcal{G}_{\text{image}}(\mathbf{I}_r^{\text{terrain}}, \mathcal{P}_r, \mathbf{I}_{\text{concept}}). \quad (9)$$

The terrain rendering and camera parameters provide explicit guidance for retaining the underlying topography and viewpoint, while the regional specification controls the categories, styles, densities, and spatial relationships of newly introduced content. The composition $\mathbf{I}_r^{\text{comp}}$ is not treated as final 3D geometry; instead, it serves as an explicit 2D layout prior that jointly constrains object appearance and spatial organization in the subsequent reconstruction and placement stages.

Object Generation. The region composition typically contains multiple objects with substantially different scales and poses. Reconstructing the complete image as a single 3D scene makes it difficult to preserve instance-level reconstruction quality, separate object boundaries, and recover accurate placements. We therefore employ text-guided SAM3 to extract individual 2D object instances from $\mathbf{I}_r^{\text{comp}}$. In addition to full-image inference, overlapping sliding-window inference improves recall across object scales. Local predictions are mapped back to the region-composition image coordinate system and then deduplicated and merged according to semantic labels and spatial overlap.

For each segmented instance i , we construct an object-centric image $\mathbf{I}_i$ and mask S_i by cropping and enlarging its corresponding region. Let A_i denote the homogeneous affine transformation from the region-composition coordinates to the object-centric image coordinates. Given the regional terrain-camera intrinsics K_t , the equivalent intrinsics after cropping are

$$\hat{K}_i = A_i K_t. \quad (10)$$

In homogeneous image coordinates, a pixel in the object-centric image can be mapped back to the region composition using A_i^{-1} . Because cropping and resizing modify only the image coordinate system, the camera extrinsics remain E_t . Recording A_i and $\hat{K}_i$ therefore preserves the original image location and its geometric relationship with the terrain camera when a small object is enlarged.

SAM3D then takes $\mathbf{I}_i$ and S_i as input and predicts an object mesh M_i , appearance attributes $\mathcal{U}_i$, a local-to-object-camera transformation T_{l2c}^i , and reconstruction-camera intrinsics K_i^o . We distinguish the cropped terrain camera from the object reconstruction camera internally estimated by SAM3D. Let $M_i^c = T_{l2c}^i M_i$ denote the reconstructed mesh in the object-camera space. Because single-view reconstruction may introduce scale deviations, we perform image-space scale calibration. Let $\mathcal{B}(\mathbf{I})$ denote the ratio between the foreground bounding-box area and the image area, and let $b_i^{\text{ref}} = \mathcal{B}(\mathbf{I}_i)$. We iteratively adjust a scale factor λ_i about the mesh center until

$$-\epsilon_i^- \leq \mathcal{B}(\Pi(K_i^o, \lambda_i M_i^c)) - b_i^{\text{ref}} \leq \epsilon_i^+, \quad (11)$$

where $\Pi(\cdot)$ denotes rendering projection, and ϵ_i^- and ϵ_i^+ control the tolerances for under- and over-projection, respectively. The asymmetric tolerance more strongly suppresses oversized reconstructions while allowing slight under-coverage caused by segmentation boundaries and single-view ambiguity. This calibration corrects the reconstructed scale within the object-camera coordinate system; the subsequent placement scale transfers the calibrated object from the object-camera coordinate system to the terrain-camera coordinate system.

Object Placement. After generating each object, we recover its 3D placement using a pair of corresponding rays from the object reconstruction camera and the regional terrain camera. We first cast a ray through the object-center pixel and intersect it with the camera-space mesh M_i^c , obtaining an object reference point P_o and its depth Z_o . The focused-image center is then mapped back to the region composition using A_i^{-1} , and a second ray is cast from the terrain camera through the mapped pixel. The nearest positive intersection with the terrain mesh M_t defines the target terrain anchor P_t and its depth Z_t . Because $\hat{K}_i = A_i K_t$ and the extrinsics remain unchanged, the ray from the focused-image center is geometrically equivalent to this inverse-mapped ray. This property allows high-resolution object reconstruction without sacrificing subsequent placement accuracy.

Under the assumptions of approximately square pixels and locally uniform perspective scaling, the initial 3D scale that preserves the apparent object size between the object-camera and terrain-camera coordinate systems is

$$s_i = \frac{Z_t}{Z_o} \frac{f_i^o}{f_i}, \quad (12)$$

where f_i^o denotes the focal length of the object reconstruction camera, while $\hat{f}_i$ is obtained directly from the equivalent intrinsics $\hat{K}_i$. Let R_i denote the relative rotation determined by the two camera extrinsics. For a mesh retained in the local coordinate system of SAM3D, the complete placement transformation is

$$T_{\text{place}}^i = \begin{bmatrix} s_i R_i & P_t - s_i R_i P_o \\ \mathbf{0}^\top & 1 \end{bmatrix} T_{l2c}^i. \quad (13)$$

The translation term maps P_o exactly to P_t . If T_{l2c}^i has already been baked into the exported mesh vertices, the rightmost T_{l2c}^i is omitted to avoid a duplicate transformation. Finally, to reduce slight floating caused by mesh discretization and single-view depth errors, we jointly search the anchor depth and isotropic scale along the terrain-camera ray while preserving the 2D projection. The search terminates when the contact ratio between the bottom object voxels and terrain reaches a threshold; otherwise, the candidate with the highest contact ratio is retained.

2.3.3 Scene Refinement

Although the preceding stages provide independent object meshes and their initial terrain-aligned placements, a regional scene may still contain inconsistent object scales or poses, insufficient local geometry or appearance quality, and object-terrain contact defects such as floating, excessive penetration, or unstable support. We therefore introduce a scene refinement agent that connects to a 3D engine such as Blender through an executable interface such as the Model Context Protocol (MCP). As shown in Fig. 3, the agent maintains a task queue consisting of diagnostic renders and status reports, and performs object refinement followed by terrain refinement.

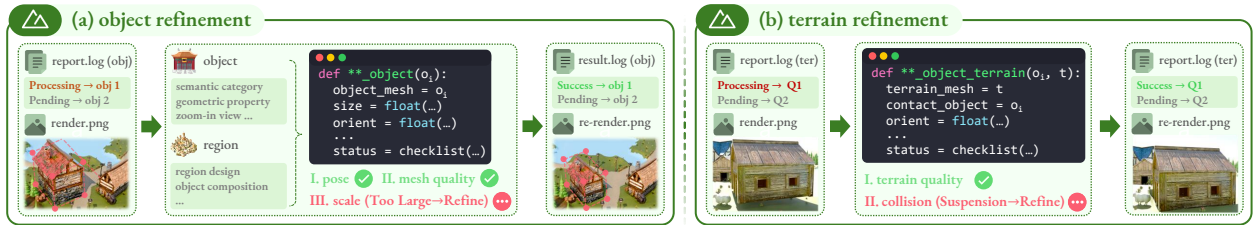


Figure 3 Scene refinement. (a) *Object Refinement* processes objects from a report queue, evaluates pose, mesh quality, and scale against semantic and regional context, applies targeted edits, and verifies the result through re-rendering. (b) *Terrain Refinement* examines support-surface quality and object-terrain collisions, applies local co-deformation to defects such as floating, and updates the report after re-rendering.

Object Refinement. As illustrated in Fig. 3(a), for each pending object, the agent evaluates pose, mesh quality, and scale using its semantic category and geometric properties together with the design and object composition of the surrounding region. Objects with inconsistent scales or implausible orientations are corrected by updating their placement transformations. For SAM3D reconstructions with insufficient geometry or appearance quality, we further condition Hunyuan3D on both the scale-calibrated coarse mesh and the object-centric image. The coarse mesh provides a structural constraint, while the image supplies shape and appearance cues, allowing Hunyuan3D [? ?] to improve local surface geometry and generate higher-quality textures or PBR materials while limiting unintended changes to object identity and overall shape. The refined asset inherits the existing T_{place}^i , and can therefore replace the coarse reconstruction without repeating placement. The agent then re-renders the processing object and updates its report until pose, mesh quality, and scale pass the corresponding checks.

Terrain Refinement. Once all objects in the pending queue have been refined, the agent proceeds to terrain refinement. As shown in Fig. 3(b), this stage evaluates terrain-surface quality together with object-terrain collision and contact, focusing on floating, excessive penetration, and unstable support. When a defect is detected, the agent performs object-terrain co-deformation within the local support region. Depending on the

object category and contact state, the object may be vertically repositioned or partially embedded, while the supporting terrain is locally displaced, flattened, or smoothed to conform to the object footprint. All modifications are restricted to the support region to preserve the global landform and neighboring terrain structures. After each edit, the agent re-renders the scene from diagnostic viewpoints and updates the terrain report according to the support-surface and collision checks. Refinement continues until the current task passes the checks or reaches the predefined iteration budget.

3 Experiments

4 Related Work

3D scene generation has attracted increasing attention in computer vision and graphics [? ? ?]. In this section, we review representative studies on static 3D scene generation and categorize existing methods into four paradigms: *procedural generation* (Section 4.1), *visual-prior-based generation* (Section 4.2), *native 3D generation* (Section 4.3), and *LLM-driven generation* (Section 4.4).

4.1 Procedural 3D Scene Generation

Procedural generation synthesizes scene content through predefined algorithms, rules, and executable programs, and has long served as a fundamental paradigm for virtual environment creation. According to the evolution of generation paradigms, existing procedural approaches can be broadly categorized into classical procedural generation and LLM-guided procedural generation.

Classical procedural generation. Classical procedural methods construct virtual environments by explicitly encoding generation rules, stochastic models, or grammar-based algorithms. They have been extensively applied to the synthesis of terrains [? ?], vegetation [? ?], and buildings and cities [? ? ?], and have become a standard content-generation technique in games, films, simulation, and virtual-world creation [? ?]. Owing to their controllability, scalability, and repeatability, procedural methods are also well suited for constructing synthetic datasets with automatically generated geometry, materials, and annotations for computer vision. Representative examples include the Infinigen series: Infinigen [?], which procedurally synthesizes natural environments containing terrains, vegetation, animals, and materials; Infinigen Indoors [?], which extends the framework to complete indoor environments; and Infinigen-Articulated [?], which further introduces articulated assets with physical properties and simulator-ready annotations.

LLM-guided procedural generation. Recent advances in large language models (LLMs) and multimodal foundation models have enabled procedural generation to be driven by natural language and visual conditions. Rather than manually specifying procedural parameters, these methods employ LLMs to interpret user intent, organize generation workflows, and coordinate procedural modules. CityX [?] and SceneX [?] utilize LLM planners and multi-agent systems to orchestrate procedural pipelines for large-scale urban scenes and natural environments. LatticeWorld [?] translates multimodal instructions into symbolic layouts and environmental configurations that are executed in Unreal Engine, while LandCraft [?] combines semantic layouts, height maps, and procedural terrain modules to synthesize structured landscapes. Compared with classical procedural generation, these methods substantially improve semantic controllability and automation. Nevertheless, scene synthesis still fundamentally depends on predefined procedural operators and handcrafted generation modules, limiting the flexibility and generality of procedural world construction.

4.2 Visual-Prior-Based 3D Scene Generation

Visual-prior-based methods leverage pretrained image or video generative models to synthesize visual observations and lift them into 3D through depth estimation, geometric reprojection, multi-view fusion, or explicit representation optimization. Based on their dominant scene-construction strategies, existing methods can be broadly organized into progressive scene expansion, global scene initialization, structured scene generation, and video-guided world generation.

Progressive scene expansion. One line of research progressively synthesizes and integrates visual observations along camera trajectories to extend scene coverage and construct an explorable 3D environment. Infinite

Nature [?], InfiniteNature-Zero [?], Persistent Nature [?], and DiffDreamer [?] primarily focus on extending natural scenes over long-range camera motion. SceneScape [?] combines text-conditioned view synthesis with monocular depth estimation to generate geometrically consistent long-term views, while WonderJourney [?] constructs sequences of diverse yet semantically connected point-cloud scenes. ScenePainter [?] further improves long-range consistency by mitigating semantic drift during successive scene expansion.

Other methods following this incremental construction paradigm place greater emphasis on integrating generated observations into a persistent scene-level representation. Text2Room [?] progressively synthesizes RGB-D observations from selected viewpoints and fuses the resulting geometry into a seamless textured mesh of an indoor environment. LucidDreamer [?] alternates between image inpainting, depth-based point-cloud expansion, and geometric alignment before optimizing the accumulated observations as a unified 3D Gaussian scene. RealmDreamer [?] similarly constructs a forward-facing 3D Gaussian representation using diffusion-based inpainting and monocular depth supervision. WonderTurbo [?] accelerates incremental geometry and appearance construction through efficient depth completion, Gaussian updates, and lightweight diffusion inpainting. Beyond individual environments, WonderWorld [?] enables low-latency, user-controlled expansion of connected 3D scenes, whereas Infinicity [?], CityDreamer [?], and GaussianCity [?] extend generative scene construction toward large-scale urban environments.

Global scene initialization. Rather than constructing a scene solely through sequential extrapolation, another line of work first establishes a globally coherent visual proxy, typically in the form of an omnidirectional panorama, and subsequently lifts it into 3D. PERF [?] represents an early attempt to reconstruct a panoramic neural radiance field from a single panorama by combining panoramic depth estimation with progressive RGB-D inpainting. More recent methods combine generative panoramic priors with efficient 3D Gaussian Splatting. FastScene [?] generates a text-conditioned indoor panorama, synthesizes additional observations through progressive inpainting, and reconstructs the environment using panoramic Gaussian splatting. DreamScene360 [?] generates a globally coherent panorama and lifts it into an unconstrained 3D Gaussian scene. HoloDreamer [?] employs a high-resolution equirectangular panorama as a holistic scene initialization and adopts a two-stage Gaussian reconstruction process to produce view-consistent, enclosed 3D worlds. SceneDreamer360 [?] further improves text-conditioned panoramic generation and introduces point-cloud-based initialization for spatially consistent panoramic Gaussian reconstruction.

Beyond direct panoramic lifting, LayerPano3D [?] and Scene4U [?] decompose panoramic observations into multiple depth or semantic layers to recover occluded content and obtain more structured scene representations. HunyuanWorld 1.0 [?] further integrates panoramic generation, geometric reconstruction, and semantic scene decomposition into a unified framework for producing explorable and structured 3D environments. Global initialization provides a more coherent scene-level prior and reduces the long-range drift associated with sequential extrapolation.

Structured scene generation. More recent methods seek to construct scenes as collections of semantically meaningful instances or independently manageable 3D components, rather than representing the entire environment as a single mesh or 3D Gaussian. General image-to-3D models such as SAM3D [?] provide open-set object reconstruction priors that can be applied to individual scene elements. MIDI [?] extends pretrained object-generation priors to jointly generate multiple 3D instances while modeling their interactions and spatial relationships. SceneTransporter [?] similarly recovers multiple scene entities and their structural arrangements from visual observations.

A modular line of research explicitly decomposes scene construction into object-level stages. SceneMaker [?] separates object de-occlusion, 3D generation, and pose estimation to improve open-set scene construction under severe occlusion. CAST [?] and related approaches recover individual objects together with their spatial configurations, while Sketch2Scene [?] extracts semantic components and layout information from a generated isometric image and instantiates the resulting scene through a procedural game-engine pipeline. Other methods incorporate hierarchical parsing, background reconstruction, and structural or physical constraints to obtain independently editable or replaceable scene components [? ?]. WorldGen [?] combines language-based layout reasoning, procedural blockouts, navigation constraints, reference-image generation, holistic reconstruction, scene decomposition, and object-level enhancement to produce traversable and editable 3D worlds compatible with standard game engines. Compared with monolithic scene representations, these methods provide stronger

support for instance-level manipulation and downstream interaction.

Video-guided world generation. Video generative models provide temporally coherent visual observations under continuous camera motion, offering denser multi-view evidence for static 3D scene construction. Scene Splatter [?] uses video diffusion to synthesize consistent novel views and iteratively refine a global 3D Gaussian representation, whereas Wonderland [?] directly predicts a 3D scene representation from camera-conditioned video latents. To enlarge the range of exploration, WorldExplorer [?] coordinates multiple autoregressive camera trajectories using persistent scene memory, while FlexWorld [?] progressively integrates generated observations through geometry-aware fusion.

4.3 Native 3D Scene Generation

Native 3D methods directly model scene geometry or its latent 3D representations without relying on lifting independently generated visual observations, thereby avoiding the geometric ambiguities and structural inconsistencies introduced by image- or video-based reconstruction. According to their scene generation strategies, existing methods can be broadly categorized into holistic scene generation, progressive scene expansion, and synchronized scene generation.

Holistic scene generation. Early native 3D methods directly synthesize complete scenes within a predefined spatial extent. DiffInDScene [?] generates global occupancy using cascaded sparse diffusion and reconstructs indoor details through overlapping TSDF blocks. XCube [?] adopts hierarchical sparse-voxel diffusion to improve scene resolution and semantic attribute capacity. Frankenstein [?] jointly decodes multiple semantic signed distance fields from a shared tri-plane representation, preserving separable indoor components, while L3DG [?] introduces diffusion in a compressed 3D Gaussian latent space for renderable room generation. For outdoor environments, CADD [?] and CymbaDiff [?] synthesize semantic scene volumes through multi-scale discrete diffusion, sparse absorbing refinement, and structured spatial sequence modeling. More recently, NuiWorld [?] represents varying numbers of scene chunks as variable-length latent sequences, enabling more flexible scene extents while still generating all chunks jointly within a single global generative process. Although these methods produce globally consistent scene representations, their generation capacity remains coupled to a predefined spatial support.

Progressive scene expansion. Progressive methods relax the fixed-scene assumption by formulating scene generation as conditional 3D continuation. Starting from a seed region, they repeatedly synthesize neighboring scene content conditioned on previously generated geometry. BlockFusion [?] constrains overlapping latent tri-planes with known scene content throughout reverse diffusion, enabling iterative extrapolation of new mesh blocks. SceneFactor [?] decomposes scene generation into semantic latent outpainting followed by geometry refinement through unsigned distance fields, while SemCity [?] progressively extends outdoor semantic voxel fields via tri-plane manipulation. NuiScene [?] instead explicitly learns conditional distributions between neighboring vector-set scene chunks, avoiding repeated diffusion resampling during inference. WorldGrow [?] further introduces object-level structured latents and performs coarse-to-fine 3D block inpainting to jointly generate geometry and appearance. GaussianGPT [?] serializes quantized 3D Gaussians into autoregressive tokens, supporting unified scene generation, completion, and continuation. Although progressive generation enables scalable scene expansion, its locally conditioned synthesis remains generation-order dependent and may accumulate layout drift and long-range semantic inconsistencies.

Synchronized scene generation. Rather than generating scene blocks sequentially, synchronized methods jointly optimize multiple overlapping regions to improve global consistency. LT3SD [?] establishes coarse global structures through latent-tree outpainting before jointly denoising overlapping latent patches at finer resolutions. TRELLISWorld [?] applies object-level 3D diffusion priors across all overlapping latent tiles and aggregates their predictions throughout the denoising process. Extend3D [?] enlarges the latent space of pretrained object models and generates larger scenes by coupling the denoising trajectories of neighboring subregions. WorldFlow3D [?] instead formulates scene generation as hierarchical flow matching from coarse geometry to fine geometry and appearance, synchronizing overlapping volumetric regions through merged velocity fields. Compared with progressive expansion, synchronized generation substantially improves global coherence and boundary consistency between neighboring regions, but typically requires the target scene extent to be specified in advance, with computation and memory consumption increasing rapidly as scene

scale grows.

4.4 LLM-Driven 3D Scene Generation

LLM-driven methods leverage large language models to interpret user intent, reason about spatial relationships, generate executable scene representations, and orchestrate downstream generation tools. According to the evolution of system capabilities, existing approaches can be broadly categorized into language-guided scene planning, hierarchical and interactive scene construction, and agentic scene generation.

Language-guided scene planning. Early work primarily employs LLMs to translate natural language into scene layouts, object relationships, or executable scene descriptions. Holodeck [?] and I-Design [?] infer object sets and spatial relationships from user instructions, followed by asset retrieval and geometric layout optimization. 3D-GPT [?] converts textual descriptions into procedural functions and parameterized scene representations, while SceneCraft [?] further compiles relational scene graphs into executable Blender Python programs. These methods enable natural language to directly participate in high-level scene planning and procedural execution, but typically rely on one-shot planning with limited iterative refinement.

Hierarchical and interactive scene construction. Subsequent research extends LLM-driven generation toward larger-scale planning, richer scene hierarchies, and visually grounded interaction. HSM [?] and OptiScene [?] improve hierarchical indoor scene organization and preference-aware layout generation. CityCraft [?], MajutsuCity [?], Yo’City [?], and MANSION [?] further expand LLM planning to city-scale environments, architectural styles, functional zoning, and multi-floor buildings. Meanwhile, SceneThesis [?], spatially contextualized VLM agents [?], and HOLODECK 2.0 [?] incorporate image guidance, structured spatial memory, and interactive editing, gradually transforming LLM-driven scene generation from one-shot layout prediction toward visually grounded and iterative scene construction. Although these methods significantly improve planning quality and interaction capability, scene generation is still largely constrained by predefined generation pipelines and limited feedback mechanisms.

Agentic scene generation. More recent work formulates 3D scene generation as an observable, executable, and self-correcting agentic process. SceneWeaver [?], SAGE [?], and SceneSmith [?] iteratively modify scenes through tool invocation, visual feedback, and physical verification. VIGA [?] adopts a code–execute–render–inspect loop for long-horizon optimization of graphics programs, while SceneCode [?] compiles user requests into executable scene-generation programs and supports local regeneration through code repair and render-based verification. By explicitly integrating reasoning, execution, observation, and feedback, these agent-based systems substantially improve the autonomy and robustness of scene generation.

5 Limitations

While WorldClaw demonstrates the potential of agentic pipelines for open-world 3D scene generation, several limitations remain.

High dependency on underlying models. WorldClaw decomposes world construction into multiple stages coordinated by heterogeneous models: large language models perform planning and procedural design, image generation models produce semantic layouts and regional object compositions, and 3D generation models reconstruct individual assets. Although this decoupling improves stage-wise controllability, it places strong demands on the generalization capabilities of the underlying models. In our experiments, current open-source language models often struggled to generate procedural terrain and materials that were both executable and consistent with user requirements. Likewise, open-source image generation models frequently failed to produce usable semantic layout maps or to preserve object appearance and pose during object-image generation and extraction. The visual quality of the final scene is also directly bounded by the 3D generation backbone, as low-fidelity geometry and textures noticeably reduce immersion. Consequently, fully validating this decoupled pipeline at the current stage still requires capable models such as Claude Opus 4.8, GPT-Image-2, Gemini 3.1 Flash Image, and Hunyuan3D.

Stability risks in code generation. Several stages of WorldClaw rely on LLM-generated programs for terrain construction, procedural material generation, asset placement, and local refinement. However,

consistently translating high-level natural-language requirements into concrete programs remains difficult. Errors in scale estimation, numerical parameters, or node connectivity directly manifest in the resulting 3D scene as inconsistent landforms, inaccurate material effects, or object layouts that deviate from the user intent, often necessitating multiple render–inspect–refine iterations. This issue is particularly evident for professional 3D creation software such as Blender, whose APIs and complex node-based workflows remain challenging for current language models. Consequently, material effects that artists can construct with sophisticated node graphs are often reduced to relatively simple approximations in generated programs.

Efficiency overhead in scene generation. To construct richly populated 3D scenes with instance-level editability, WorldClaw generates and reconstructs individual objects separately and performs multiple rounds of agentic refinement over the terrain, assets, and their contact relationships. This long-horizon pipeline incurs substantial inference latency and computational cost, both of which increase with the number of objects and refinement iterations. Although this overhead provides additional editability for complex scenes that require fine-grained control, the pipeline can be unnecessarily lengthy and inefficient for simpler scenes that holistic generation methods can synthesize in fewer steps. This efficiency limitation becomes more pronounced when constructing larger or denser 3D worlds.

6 Conclusion

In this work, we presented **WorldClaw**, a coarse-to-fine agentic framework for generating explicit, explorable, and editable 3D worlds from open-ended text prompts. Its central design decouples global world organization from local instance-level content generation. WorldClaw first translates user intent into a structured scene specification and establishes a global terrain with explicit regional semantics and expressive landform hierarchy. It then selectively generates and places independent 3D objects only in regions that require fine-grained content, enriching local scene composition without disrupting the established global structure. Render-guided agents further correct terrain appearance, object quality, spatial arrangements, and object–terrain contacts. The resulting worlds consist of explicit terrain and independently manageable textured meshes, supporting free-viewpoint exploration, object-level editing and reuse, and integration with conventional rendering, animation-authoring, and game-engine workflows. Our experiments demonstrate that WorldClaw can construct 3D worlds with diverse landforms, clear regional organization, and rich object composition, achieving a practical balance among scene scale, visual quality, and editability.

The current framework still relies primarily on generative 3D models to reconstruct regional objects. Although these models provide substantial visual diversity, they do not consistently recover explicit part hierarchies, parametric structures, articulation, or interaction logic. As foundation models continue to improve in code generation and tool use, code-native 3D modeling offers a complementary direction for addressing these limitations. Future work may replace selected object-generation stages with executable modeling programs or combine programmatic structure construction with generative geometry and appearance priors. Such hybrid representations could preserve the visual diversity of open-ended generation while introducing stronger structural control and support for animation, physical interaction, and behavioral logic, extending WorldClaw from static, editable scene generation toward more interactive and playable 3D worlds.

7 Contributors

Project Leaders: Chunchao Guo, Yang Li

Core Contributors:

- **Local Planning:** Jinpeng Li, Yang Li, Zilong Huang
- **Terrain Generation:** Zilong Huang